Project 2 - Graph Database Design and Cypher Query

Lukas Rohwer (24215157)

Introduction

This report details the comprehensive design and implementation of a graph database used to analyse aviation flight data. The main considerations that I took into account when designing my graph database were ensuring that the design would allow any relevant business questions to be answered without over complicating the structure, and ensuring the design allowed for efficient graph traversal, as queries could involve a large number of records.

To achieve this, the report outlines the project through the following key phases:

Graph Database Design – I outline the final chosen schema, defining dedicated nodes representing the physical airports and individual airlines, connected by a directed ROUTE relationship.

ETL Process – I document the two-stage execution of the ETL pipeline, detailing how I utilized Python for data cleaning and transformation, and subsequently used Cypher queries to load the data into the Neo4j database.

Cypher Queries – Following the completion of the database, I demonstrate its analytical capabilities by querying the database using the five business questions detailed by the task sheet, as well as two of my own custom queries.

Concepts and Applications – Finally, I discuss the practical applications of graph data science compared to traditional relational databases, and explore how metadata is an important consideration when designing a graph database schema and writing efficient queries.

Graph Database Design

The main considerations that I took into account when designing my graph database were the following:

- The design should allow any relevant business questions to be answered without over complicating the design.
- The design should allow for efficient graph traversal as queries may involve a large number of records.

In considering the above, here is the final chosen schema that I chose.

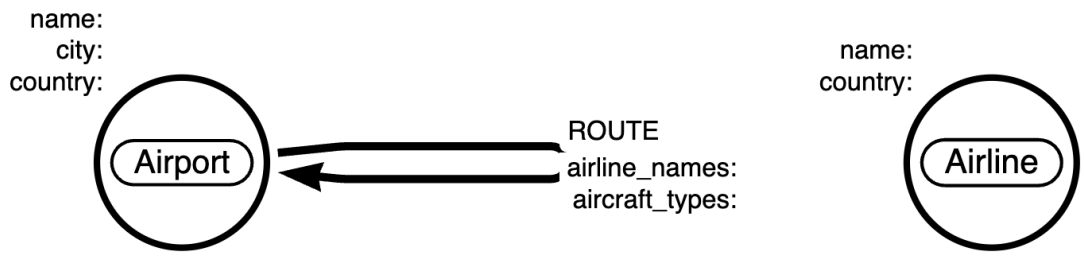
Nodes:

- **Airport**: Representing the physical airports, with the properties name, city, and country. Every airport regardless of whether it is listed as a departure or arrival airport is stored on this node. Storing the city and country directly on the node allows for highly efficient queries without traversing unnecessary relationships.
- **Airline**: Representing the individual airlines with the properties name and country. Having a dedicated node for airports is important as it reduces redundancy through a simple query, rather than only storing the information in a relationship.

Relationships

- **ROUTE**: A directed relationship the Airport node to Airport node. The direction of this relationship is what determines whether an airport is a departure airport or an arrival airport. The relationship has the properties airlines and aircraft_types, which are both stored as lists of strings stored directly on the relationship.
- **AIRLINE-ROUTE**: This is rather an implicit relationship linking airline to ROUTE using the name of the airline as the key. This is important to mention as it allows for various important queries needed to answer business questions.

Below is the visual representation of the schema described above, that I designed using the Arrow app.



While the schema design I chose prioritises query efficiency and structural simplicity, it does introduce a specific disadvantage regarding data redundancy and update management. Because the design intentionally separates the Airline entity from the Airport entity, placing the airline name as a property directly onto the ROUTE relationship, any changes to an airline's overarching details (such as a airline name) would require scanning and updating every single ROUTE edge that references that airline.

ETL Process

I executed the ETL process in two stages: Python for data cleaning and transformation, and Cypher queries to load the data into the database.

After downloading the data set from the task sheet, I loaded the data set into python using the pandas module. In my initial data exploration, I searched for null data as well as for anomalies in the data. Using `df.info()` as well as through visual inspection, I found no rows containing null data.

However, I did find a case where `Albany Airport` was listed as an airport in Australia as well as the United States. While there exists an Albany International Airport in the US, given the associated departure airport located in Australia as well, I decided that the country was entered incorrectly and could be changed back to Australia. Upon searching the data set for other records like this, I found 41 airports in total that had conflicting country entries. For each of these, I verified the correct country using Wikipedia and used a dictionary in python to correct each of these entries.

```
for airport, correct_country in corrections.items():

    df.loc[
        df['Departure Airport Name'].str.startswith(airport, na=False),
        'Departure Airport Country/Region'
    ] = correct_country

    df.loc[
        df['Arrival Airport Name'].str.startswith(airport, na=False),
        'Arrival Airport Country/Region'
    ] = correct_country
```

In my final step of data cleaning, I discovered a row that listed the departure airport and the arrival airport as the same airport. It does not make logical sense that this is considered a route, as such, it was dropped from the data set. No other rows like this existed, which I verified through python.

Next, I transformed the dataset into three separate CSV files for efficient loading into Neo4J. This process eliminates data redundancy and closely reflects the design outlined at the start of this document.

To extract the data for the Airport node, I extracted both all departure and arrival airports. I renamed the columns to align with proper database design practices and to match my design. Then I combined all of the airports into one long data frame, ensuring to drop all duplicates so that each Airport is listed correctly, exactly once. This data frame was saved to a CSV file named ‘airports_nodes.csv’.

```python
dep_airports = df[['Departure Airport Name', 'Departure Airport City', 'Departure Airport Country/Region']].rename(columns={
    'Departure Airport Name': 'name',
    'Departure Airport City': 'city',
    'Departure Airport Country/Region': 'country'
})

arr_airports = df[['Arrival Airport Name', 'Arrival Airport City', 'Arrival Airport Country/Region']].rename(columns={
    'Arrival Airport Name': 'name',
    'Arrival Airport City': 'city',
    'Arrival Airport Country/Region': 'country'
})

airports_df = pd.concat([dep_airports, arr_airports]).drop_duplicates(subset=['name', 'city', 'country']).dropna(subset=['name'])

airports_df.to_csv('airports_nodes.csv', index=False)
```

Next, I extracted the data for the Airline node. I renamed the columns in a similar way as was done to the airports, to align with the design for my database. Then, I deleted duplicates so that each airline is listed only once and saved the data to a CSV file names 'airline_nodes.csv'.

```python
airlines_df = df[['Airline Name', 'Airline Country']].rename(columns={
    'Airline Name': 'name',
    'Airline Country': 'country'
}).drop_duplicates(subset=['name']).dropna(subset=['name'])

airlines_df.to_csv('airlines_nodes.csv', index=False)
```

Finally, I extracted the data for the ROUTE relationship which links a departure and an arrival airport, with two lists each of the airlines and aircraft types that fly this route. To achieve this, I grouped the dataset by departure and arrival airport, and aggregated the airlines and aircraft types on each pairing, dropping any duplicate airlines/aircraft types on any given route. Then I renamed the columns to match my database design and saved the data to my third CSV file 'routes_edges.csv'.

```python
# group and aggregate
routes_df = df.groupby(['Departure Airport Name', 'Arrival Airport Name']).agg(
    airlines=('Airline Name', get_unique_airlines),
    aircraft_types=('Plane Name', get_unique_planes)
).reset_index()

routes_df.rename(columns={
    'Departure Airport Name': 'departure_airport',
    'Arrival Airport Name': 'arrival_airport'
}, inplace=True)

routes_df.to_csv('routes_edges.csv', index=False)
```

Following the data extraction and transformation, I was ready to load the data into Neo4j using Cypher queries.

Upon the creation of the database, the first query I executed was the following:

CREATE CONSTRAINT FOR (a:Airport) REQUIRE a.name IS UNIQUE;

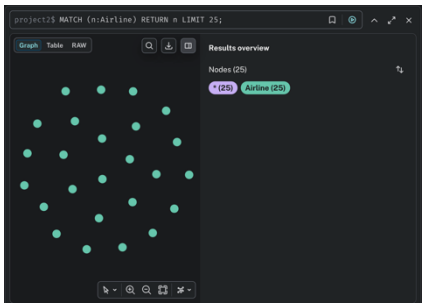
```
CREATE CONSTRAINT FOR (air:Airline) REQUIRE air.name IS UNIQUE;
```

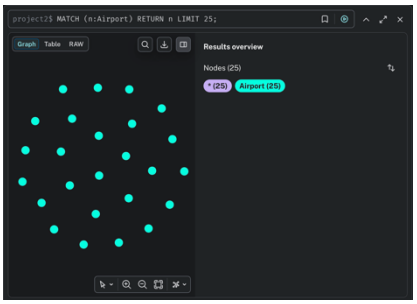
This ensures that airline and airport names are unique identifiers, which I intentionally designed throughout my data transformation. This allows for efficient and simple queries in the future, without having to worry about potential conflicts causing crashes.

Following this, the next query I ran was:

```
LOAD CSV WITH HEADERS FROM 'file:///airlines_nodes.csv' AS row
MERGE (air:Airline {name: row.name})
SET air.country = row.country;
```

This query loads the data from the CSV file that I created, ensuring without a doubt that each row contains a unique airline, with the properties name and country.





Similarly, I loaded in the data for the airports node, again ensuring that each row contains a unique airport, using the following query.

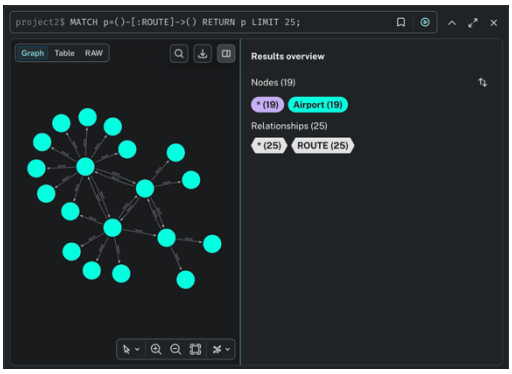
```
LOAD CSV WITH HEADERS FROM 'file:///airports_nodes.csv' AS row
MERGE (a:Airport {name: row.name})
SET a.city = row.city,
   a.country = row.country;
```

Finally, I loaded in the 'route_edges.csv' file and created the ROUTE relationship between airports using the following query:

```
LOAD CSV WITH HEADERS FROM 'file:///routes_edges.csv' AS row
MATCH (dep:Airport {name: row.departure_airport})
MATCH (arr:Airport {name: row.arrival_airport})
MERGE (dep)-[r:ROUTE]->(arr)
SET r.airlines = split(row.airlines, ';'),
```

```
r.aircraft_types = split(row.aircraft_types, ';');
```

Using MATCH, the query searches the existing Airport nodes for the matching airports and then draws the ROUTE relationship between them using MERGE. Finally, the two list properties are assigned to the relationship, completing the database.



Cypher Queries

Following the completion of the database, I was able to begin querying the database using the five business questions detailed by the task sheet, as well as two of my own.

Query 1

List all distinct airline names where the airline's country is Australia.

```
MATCH (air:Airline {country: 'Australia'})
RETURN DISTINCT air.name AS AirlineName;
```

AirlineName
1 "Whyalla Airlines"
2 "Transpac Express"
3 "Transaustralian Air Express"

There are 3 airlines that have 'Australia' recorded as their country: Whyalla Airlines, Transpac Express, Transaustralian Air Express.

Query 2

How many route records in the dataset are domestic, and how many are international?

```
MATCH (dep:Airport)-[r:ROUTE]->(arr:Airport)
RETURN
  SUM(CASE WHEN dep.country = arr.country THEN 1 ELSE 0 END) AS DomesticRecords,
```

```
  SUM(CASE WHEN dep.country <> arr.country THEN 1 ELSE 0 END)
AS InternationalRecords;
```

DomesticRecords	InternationalRecords
14418	18069

There are 14418 unique routes that fly domestically (within country) and 18069 unique routes that fly internationally (between countries).

Query 3

Find the airport pair with the greatest number of services operated by different airlines.

```
MATCH (dep:Airport)-[r:ROUTE]->(arr:Airport)
WITH
  CASE WHEN dep.name < arr.name THEN dep.name ELSE arr.name
END AS Airport1,
  CASE WHEN dep.name < arr.name THEN arr.name ELSE dep.name
END AS Airport2,
  r.aircraft_types AS planes
UNWIND planes AS plane
RETURN Airport1, Airport2, count(DISTINCT plane) AS
DistinctAircraftCount
ORDER BY DistinctAircraftCount DESC
LIMIT 5;
```

Airport1	Airport2	RecordCount
"Chicago O'Hare International Airport"	"Hartsfield Jackson Atlanta International Airport"	19

The route serviced by the most airlines is the route between Chicago O'Hare International Airport to Hartsfied Jackson Atlanta International Airport, which is serviced by 19 individual airlines.

Query 4

Find the top 5 airport pairs that are served with the greatest number of distinct aircraft types across all rows and all airlines.

```
MATCH (dep:Airport)-[r:ROUTE]->(arr:Airport)
WITH
  CASE WHEN dep.name < arr.name THEN dep.name ELSE arr.name END AS Airport1,
  CASE WHEN dep.name < arr.name THEN arr.name ELSE dep.name END AS Airport2,
  r.aircraft_types AS planes
UNWIND planes AS plane
RETURN Airport1, Airport2, count(DISTINCT plane) AS DistinctAircraftCount
ORDER BY DistinctAircraftCount DESC
LIMIT 5;
```

	Airport1	Airport2	DistinctAircraftCount
1	"Hong Kong International Airport"	"Incheon International Airport"	15
2	"Hong Kong International Airport"	"Suvarnabhumi Airport"	13
3	"Beijing Capital International Airport"	"Guangzhou Baiyun International Airport"	13
4	"Incheon International Airport"	"Kansai International Airport"	12
5	"Beijing Capital International Airport"	"Shanghai Hongqiao International Airport"	12

These are the top five routes serviced by the largest number of distinct aircraft types, with the route between Hong Kong International Airport and Incheon International Airport being serviced by the most, 15.

Query 5

Find all possible travel routes from Beijing Capital International Airport to Perth International Airport where at most 3 hops (i.e., at most 3 ROUTE relationships) are traversed. Treat each unique sequence of airports visited as a distinct route. How many such distinct routes exist?

```
MATCH path = (start:Airport {name: 'Beijing Capital International Airport'})-[:ROUTE*1..3]->(end:Airport {name: 'Perth International Airport'})

WITH [a IN nodes(path) | a.name] AS airportSequence

RETURN count(DISTINCT airportSequence) AS NumberOfDistinctRoutes;
```

NumberOfDistinctRoutes
1 650

The are 650 distinct routes between Beijing Capital and Perth Internation Airport, going through at most 3 routes.

Query 6 (Custom)

Find the top 5 airports that fly directly to the most distinct foreign countries.

This query leverages the country property stored directly on the airport node to measure global connectivity rather than the raw number of flights.

```
MATCH (dep:Airport)-[:ROUTE]->(arr:Airport)

WHERE dep.country <> arr.country

RETURN

  dep.name AS HubAirport,

  dep.country AS HubCountry,

  count(DISTINCT arr.country) AS DistinctForeignCountriesReached

ORDER BY DistinctForeignCountriesReached DESC

LIMIT 5;
```

This query works by first matching all routes except those that connect two airport nodes with the same country property. This filters out all domestic routes leaving only international routes. It then aggregates the results by the airports' name and country and counts the distinct number of countries connected to that airport. The results are ordered by number of distinct of countries from highest to lowest, limited to 5 results.

	HubAirport	HubCountry	DistinctForeignCountriesReached
1	"Charles de Gaulle International Airport"	"France"	103
2	"Istanbul Airport"	"Turkey"	90
3	"Frankfurt am Main Airport"	"Germany"	89
4	"Dubai International Airport"	"United Arab Emirates"	76
5	"Amsterdam Airport Schiphol"	"Netherlands"	76

The five airports listed above fly directly to the most distinct foreign countries, acting as international gateways. What is visible is that major airports in Europe reach a large number of countries, since Europe is so densely packed with countries. This is especially true for Paris, as one of the most popular tourist destinations in the world. Dubai International Airport is expected to be on this list as one of the busiest airports in the world by passenger traffic, acting as a large international hub.

Query 7 (Custom)

Find the top 10 aircraft types that are flown over the greatest number of routes.

This query leverages the apoc library to flatten the arrays of aircraft types stored on the ROUTE relationship and count them in an efficient way.

```
MATCH ()-[r:ROUTE]->()
WITH collect(r.aircraft_types) AS nested_planes
WITH apoc.coll.flatten(nested_planes) AS all_planes
UNWIND apoc.coll.frequencies(all_planes) AS plane_data
RETURN
  plane_data.item AS AircraftType,
  plane_data.count AS TotalRouteAppearances
ORDER BY TotalRouteAppearances DESC
LIMIT 10;
```

The query first matches all routes in the database and collects every list of aircraft types. Then using apoc.coll.flatten() the lists are flattened into one large one-dimensional array of aircraft types, and the frequencies are calculated using apoc.coll.frequencies(). The query returns these counts ordered by number of appearances descending and limits the number of records to 10.

	AircraftType	TotalRouteAppearances
1	"Airbus A320"	10473
2	"Boeing 737-800"	7854
3	"Airbus A319"	5802
4	"Airbus A321"	2674
5	"Boeing 737"	2086

The above 5 aircraft types are the most utilized aircraft types across the globe, noticeably the Airbus A320 which flies over ~33% more routes than the second most utilized aircraft, the Boeing 737-800. These two aircrafts dominate the aviation industry proving that the industry relies on high-frequency regional and domestic flights rather than long haul flights, which is what these short to medium haul aircraft are designed for.

Practical Applications of Graph Data Science

Rather than using rigid tables, rows, and columns as seen in traditional relational databases, graph databases are built entirely around the concepts of nodes, relationships, and properties. As a result, they offer various benefits over regular relational databases including:

1. Efficient handling of highly connected data – relationships are stored as direct pointers and can be traversed simple pointer hops rather than using multiple expensive JOIN operations.
2. Flexible schema – rigid tables, rows, and columns make it difficult to add new relationships or entities. Graph databases are inherently flexible making it easy to add new relationships or nodes.
3. Intuitive design – Rather than having to comprehend the structure of related through complex tables, graph databases consist of nodes connected by edges making them much easier to visualise and understand at a glance.

However, traditional relational databases still have their place in handling bulky data and tabular reporting over largely disconnected data.

Two examples of practical applications where graph data science is commonly used include the following [4]:

1. Peer-to-peer networks – Protocols such as BitTorrent are essentially a massive dynamic graph. Nodes represent peers in the network, and they are connected

by edges (relationships). These protocols use the breadth first search algorithm to discover all neighbouring nodes of a node, allowing peers to connect to the nearest nodes in the network.

2. Maps and directions – Applications such as google maps are some of the largest graph databases in the world. Nodes are intersections and edges are the roads in between them. Furthermore, each node is associated with a weight: the time to travel that road. Utilising Dijkstra's algorithm, applications such as google maps can path find the route from one node to another with the lowest weight aka. with the least amount of time to travel allowing users to take the fastest route to their destination.

Purpose of Metadata in Graph Databases

Graph databases existing in graph database management systems such as Neo4j store important information about the structure of the graph database itself through meta data such as node labels, relationship types, and property keys.

As such, it is crucial to not only include metadata when designing a graph database, but also to implement metadata in a way to maximise both efficiency and data quality. Metadata refers to the structure of the database, including node labels (e.g. Airport, Airline), relationship types (e.g. ROUTE), and property keys (e.g. city, aircraft_types). While graph databases are technically schema-optional (meaning you can add any property to any node at any time), it is important to define a schema through metadata and maintain it through the lifecycle of the database for the following reasons:

1. Data Integrity – By defining a specific schema with labels and property keys, you can apply specific uniqueness and existence constraints on the data. For example, in the database described in this report, I defined a uniqueness constraint on the name property on the Airport node when loading in the data. Without metadata, I would not be able to reference specific properties and maintaining this integrity would be impossible.
2. Modelling strategy – Understanding the metadata limitations of a graph database can directly influence the design choices in the planning stage. For example, knowing that relationships cannot be between a node and another relationship informed my decision to map the Airline name as a property on the ROUTE relationship.
3. Maintenance – If the metadata of a graph data base is logical designed and well-documented, databases can easily be maintained in the future, even in the case of a change of ownership. If the database needs to be updated in the future, adjustments can easily be made guided by the initial design choices laid out by the metadata.

Furthermore, thoughtful design through metadata can even be leveraged to write more effective and efficient Cypher queries. Tools such as 'apoc.meta.schema()' or 'db.schema.visualization()' allow you visually see the graph's metadata laid out in front of you, providing insight into the dataset without having to read low-level files or documentation. This would allow a new developer to the project to instantly understand the relationships between the data, for example they would be able to see that there is no direct relationship between Airline and Airport. Furthermore, calling 'apoc.meta.schema() [5]' would even allow them to understand the properties of nodes and relationships, for example, they would be able to see that aircraft_types is a list of strings under the ROUTE relationship. Beyond improving visibility, metadata can also be used to optimise queries through specifying node labels or relationship types meaning that Neo4j can restrict the search space rather that searching the entire database, significantly reducing the computing required to execute the query. Finally, specific commands can also use metadata to write advanced queries that dynamically adapt to the structure of the graph. For example, using the call 'db.relationshipTypes()' you can search the database for all relationship types and then generate an output that describes each relationship it finds.

Conclusion

In conclusion, this project successfully demonstrates the robust design and implementation of a graph database to analyze global aviation networks. By designing the schema with dedicated Airport and Airline nodes connected by a directed ROUTE relationship , I ensured that the structure could efficiently answer relevant business questions without overcomplicating the design.

Using a two stage ETL process leveraging Python and Cypher queries was important in establishing a reliable database. Through data exploration and cleaning, I was able to resolve geographical anomalies where airports had conflicting country entries and transform the raw dataset to eliminate data redundancy. The subsequent execution of standard and custom Cypher queries, which evaluated everything from domestic route counts to global aircraft popularity, proved the powerful analytical capabilities of the final data model.

Furthermore, the theoretical exploration of graph data science highlighted the benefits of graph databases over traditional relational databases, particularly in their flexible schema and efficient handling of highly connected data. Finally, the project displayed the importance of metadata in maintaining data integrity, guiding modeling strategies, and optimizing query performance. Overall, this report validates how a thoughtfully designed property graph can transform raw, disconnected flight records into highly accessible and actionable insights.

Bibliography

[1] neo4j, “Import CSV data using LOAD CSV,” [Online]. Available: https://neo4j.com/docs/getting-started/data-import/csv-import/.

[2] L. X. i. Y. minutes, “Where X=Cypher,” [Online]. Available: https://learnxinyminutes.com/cypher/.

[3] A. Frame, “Graph Data Science: The Secret to Accelerating Innovation with AI/ML,” 26 1 2022. [Online]. Available: https://neo4j.com/blog/graph-data-science/graph-data-science-the-secret-to-accelerating-innovation-with-ai-ml/.

[4] Memgraph, “Memgraph,” 11 04 2022. [Online]. Available: https://memgraph.com/blog/graph-algorithms-applications.

[5] neo4j, “What is graph data modeling?,” [Online]. Available: https://neo4j.com/docs/getting-started/data-modeling/.